Aula 13 – Padrões de Projeto (Design Patterns) com C#

Singleton, Factory, Observer e MVC

Professor: Eduardo Luan Pilonetto

Agenda da Aula

- Introdução aos Design Patterns
- Problemas comuns em arquitetura de software
- Padrão Singleton
- Padrão Factory
- Padrão Observer
- Padrão MVC
- Boas práticas e princípios SOLID
- Exercícios práticos
- Projeto final integrado

O que são Design Patterns?

- Soluções reutilizáveis para problemas recorrentes em software.
- Criados para melhorar manutenção e escalabilidade.
- Não são código pronto, mas modelos de solução.
- Popularizados pelo livro 'Gang of Four (GoF)'.

Benefícios dos Patterns

- Redução de acoplamento.
- Maior reutilização de código.
- Facilidade de manutenção.
- Melhor comunicação entre desenvolvedores.
- Arquitetura mais organizada.

Categorias de Patterns

- Criacionais → criação de objetos.
- Estruturais → organização de classes.
- Comportamentais → comunicação entre objetos.
- Singleton e Factory → Criacionais.
- Observer → Comportamental.
- MVC → Arquitetural.

Singleton – Conceito

- Garante apenas uma instância de uma classe.
- Fornece ponto global de acesso.
- Muito usado em logs, configuração e conexão.
- Evita múltiplas instâncias desnecessárias.

Problemas resolvidos pelo Singleton

- Controle centralizado de recursos.
- Evita inconsistência de estado.
- Economia de memória.
- Controle único de configuração.

Singleton em C#

```
public sealed class Logger
{
    private static Logger instancia;

    private Logger() { }

    public static Logger Instancia
    {
        get
        {
            if (instancia == null)
            {
                instancia = new Logger();
            }
            return instancia;
        }
    }

    public void Escrever(string msg)
    {
        Console.WriteLine(msg);
    }
}
```

A classe possui construtor privado e apenas uma instância compartilhada.

Vantagens e Desvantagens do Singleton

- Vantagens: simplicidade, controle global, economia de recursos.
- Desvantagens: alto acoplamento.
- Pode dificultar testes unitários.
- Uso excessivo pode virar antipattern.

Factory – Conceito

- Centraliza criação de objetos.
- Oculta detalhes de instanciação.
- Facilita extensão do sistema.
- Reduz dependência direta.

Quando usar Factory?

- Quando há múltiplos tipos de objetos.
- Quando criação é complexa.
- Quando deseja desacoplamento.
- Quando há regras de negócio na criação.

Factory em C#

```
public interface IVeiculo
{
    void ExibirTipo();
}

public class Carro : IVeiculo
{
    public void ExibirTipo()
    {
        Console.WriteLine("Carro");
    }
}

public class Moto : IVeiculo
{
    public void ExibirTipo()
    {
        Console.WriteLine("Moto");
    }
}

public class VeiculoFactory
{
    public static IVeiculo Criar(string tipo)
    {
        if(tipo == "carro")
            return new Carro();

        return new Moto();
    }
}
```

A fábrica decide qual objeto será criado conforme a necessidade.

Benefícios do Factory

- Reduz dependência entre classes.
- Facilita manutenção.
- Permite adicionar novos tipos facilmente.
- Melhora escalabilidade.

Observer – Conceito

- Permite comunicação entre objetos.
- Quando um objeto muda, outros são notificados.
- Muito usado em eventos e interfaces gráficas.
- Modelo Publish/Subscribe.

Exemplos de uso do Observer

- Sistema de notificações.
- Eventos em interfaces gráficas.
- Monitoramento em tempo real.
- Atualização automática de dashboards.

Observer em C#

```
public interface IObserver
{
    void Atualizar(string mensagem);
}

public class Usuario : IObserver
{
    public void Atualizar(string mensagem)
    {
        Console.WriteLine(mensagem);
    }
}

public class Canal
{
    private List<IObserver> inscritos = new();

    public void Inscrever(IObserver obs)
    {
        inscritos.Add(obs);
    }

    public void Notificar(string msg)
    {
        foreach(var obs in inscritos)
        {
            obs.Atualizar(msg);
        }
    }
}
```

Os observadores recebem atualizações automaticamente.

Vantagens do Observer

- Baixo acoplamento.
- Facilidade de extensão.
- Comunicação eficiente.
- Ideal para eventos.

MVC – Conceito

- Padrão arquitetural.
- Separa responsabilidades.
- Model → dados e regras.
- View → interface.
- Controller → controle do fluxo.

Fluxo do MVC

- Usuário interage com View.
- Controller recebe requisição.
- Model processa regras.
- Resultado retorna para View.

MVC em C#

```
public class Produto
{
    public string Nome { get; set; }
}

public class ProdutoController
{
    public Produto ObterProduto()
    {
        return new Produto { Nome = "Notebook" };
    }
}
```

O Controller controla o acesso aos dados e envia para a View.

Vantagens do MVC

- Separação de responsabilidades.
- Facilidade de manutenção.
- Maior organização.
- Ideal para aplicações web.

Comparação entre os Patterns

- Singleton → única instância.
- Factory → criação flexível.
- Observer → comunicação entre objetos.
- MVC → organização arquitetural.
- Cada padrão resolve um problema diferente.

Relação com SOLID

- Patterns ajudam aplicação de SOLID.
- Factory reduz dependência (DIP).
- Observer favorece OCP.
- MVC melhora SRP.
- Singleton deve ser usado com cuidado.

Projeto Integrador da Aula

- Sistema de e-commerce.
- Singleton para configurações.
- Factory para meios de pagamento.
- Observer para notificações.
- MVC para organização geral.
- Objetivo: integrar todos os padrões.

Conclusão

- Design Patterns resolvem problemas recorrentes.
- Melhoram arquitetura e manutenção.
- Devem ser usados com critério.
- Conhecimento essencial para desenvolvedores C#.

Perguntas e Discussão

- Dúvidas sobre Singleton?
- Quando usar Factory?
- Observer versus eventos tradicionais?
- MVC ainda é relevante?